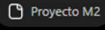


me desis si esta bien el test o falta algo para agregar

Sí. Revisé completo tu archivo, especialmente la sección `src/test/server.test.js`. En general el test está bastante bien y cubre la mayor parte del CRUD de autores y posts, pero hay algunas pruebas que te conviene agregar para que quede más completo y sólido para la evaluación. 

✓ Lo que ya tenés cubierto

Health y Swagger

- `/health` → 200
- `/api-docs.json` → 200
- OpenAPI 3.0.0

Autores

- GET todos → 200
- GET inexistente → 404
- POST correcto → 201
- POST sin email → 400
- POST email inválido → 400
- POST email repetido → 400
- PUT correcto → 200
- PUT email inválido → 400
- PUT ID inválido → 400
- PUT autor inexistente → 404
- DELETE correcto



un codigo error 204 puede devolver un mensaje de una DELETE por ejemplo el autor a sido eliminado

No. HTTP 204 No Content no debe devolver un mensaje ni ningún cuerpo (`body`).

Por ejemplo, esto **no es correcto**:

```
</> JavaScript   
res.status(204).json({  
  msg: "El autor ha sido eliminado"  
});
```

Aunque Express pueda intentar procesarlo, un `204` significa literalmente que la respuesta **no tiene contenido**.

Tenés dos opciones correctas

Opción 1 — usar 204:

```
</> JavaScript   
res.status(204).send();
```

El cliente recibe `204` y sabe que la eliminación fue exitosa, pero **sin mensaje**.

mira este es mi proyecto del m2 y queria tener el alcance minimo y me anda de 10 pero tengo duda en la respuesta con codigo http adecuado 200,201,204,400,404,500 me ayuda a ver , asi lo cambio si hay alguno que no esta bien este es lo que aparece en la guia: 200 OK - Éxito genérico
201 Created - Recurso creado exitosamente
204 No Content - Éxito pero sin body (común en DELETE)

400 Bad Request - Request malformada o inválida
404 Not Found - Recurso no existe

Ver más ▾

Sí. Revisé tu proyecto completo y, comparándolo con la guía que vos pasaste, en general los códigos HTTP que estás usando están bien. Pero encontré algunos puntos concretos que conviene corregir, sobre todo en la documentación Swagger para que coincida con lo que realmente devuelve tu API.

1. Lo que tenés actualmente

Endpoint	Situación	Código correcto
GET /authors	Lista autores	200 ✓
GET /authors/:id	Autor encontrado	200 ✓
GET /authors/:id	Autor inexistente	404 ✓
POST /authors	Autor creado	201 ✓
POST /authors	Datos inválidos / email repetido	400 ✓

Swagger UI

ChatGPT

projectom2/marciano-csae-3a3-production.up.railway.app/api/docs

Request URL

https://projectom2/marciano-csae-3a3-production.up.railway.app/api/authors

Server response

Code

Details

200

Response body

```
{
  "data": [
    {
      "id": 1,
      "name": "George Orwell",
      "email": "george@orwell.es",
      "role": "Novelista y ensayista británico, reconocido por",
      "created_at": "2014-01-11T09:30:00Z"
    },
    {
      "id": 2,
      "name": "Mary Shelley",
      "email": "mary@shelley.org",
      "role": "Novelista y ensayista británica, pionera de la",
      "created_at": "2014-01-11T11:00:00Z"
    },
    {
      "id": 3,
      "name": "Amel El Sassi",
      "email": "amel@el-sassi.com",
      "role": "test"
    },
    {
      "id": 4,
      "name": "cesar",
      "email": "cesar@gmail.com",
      "role": "test"
    }
  ]
}
```

Response headers

modulo2(2).txt

Documento

por que cuando ingreso los datos el id y created_at no se agregan de manera automatica

entonces necesitás que la tabla `authors` tenga algo parecido a:

`</>` SQL



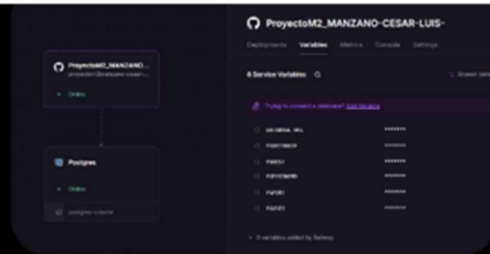
```
CREATE TABLE authors (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL,  
  bio TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Y después el controlador debería hacer el `INSERT` en PostgreSQL, por ejemplo:

`</>` JavaScript



```
const postaddauthor = async (req, res) => {  
  try {  
    const { name, email, bio } = req.body;  
  
    const result = await pool.query(  
      `INSERT INTO authors (name, email, bio)  
      VALUES ($1, $2, $3)  
      RETURNING *`,  
      [name, email, bio]  
    );  
  
    res.status(201).json({  
      msg: "Autor agregado correctamente",  
      data: result.rows[0]  
    });  
  
  } catch (error) {  
    console.error(error);  
  
    res.status(500).json({  
      msg: "Error al crear el autor"  
    });  
  }  
};
```



el problema para mí está en el Railway porque mi código sí funciona de 10 cuando lo pruebo localmente:

Sí, tu sospecha tiene bastante sentido. Si localmente `GET /comments` funciona perfectamente y en Railway devuelve `500`, entonces hay que revisar principalmente la conexión y el estado de la base de datos de Railway, no tu controlador.

Y en tu captura veo algo importante: tu servicio de Node tiene las variables `DATABASE_URL`, `PGDATABASE`, `PGHOST`, `PGPASSWORD`, `PGPORT` y `PGUSER`, así que las variables sí están llegando al servicio.

Pero que las variables existan no significa que la base de datos de Railway tenga las mismas tablas/datos que tu PostgreSQL local.

La prueba que nos va a sacar de dudas

Entrá en:

Railway → PostgreSQL → Console

y ejecutá:



```
-- Crear la base de datos
CREATE DATABASE blog_db;
-- Crear usuario específico para la aplicación
CREATE USER blog_user WITH PASSWORD 'blog_password_2026';
-- Conectar a la base de datos
\c blog_db
-- Dar permisos al usuario en la base de datos
GRANT ALL PRIVILEGES ON DATABASE blog_db TO blog_user;
-- Dar permisos en el schema public
GRANT ALL PRIVILEGES ON SCHEMA public TO blog_user;
Ver más ▾
```

Perfecto. Ahora veo exactamente qué pasó. Tu script local no se puede ejecutar completo tal cual en Railway, porque allí la base de datos `railway` ya existe y Railway administra el usuario y la conexión.

La buena noticia es que no necesitás crear otra base de datos ni otro usuario. Solo tenemos que crear las tablas dentro de la base de datos de Railway.

1. No ejecutes estas partes en Railway

No ejecutes:

SQL

```
CREATE DATABASE blog_db;
```

4. Cargamos los datos

Después podés ejecutar tus inserts.

Autores

SQL

```
INSERT INTO authors (name, email, bio) VALUES
('Ana García', 'ana@example.com', 'Desarrolladora full-stack apasionada por Node.js'),
('Carlos Ruiz', 'carlos@example.com', 'Escritor técnico especializado en bases de datos'),
('María López', 'maria@example.com', 'Ingeniera de software con foco en APIs REST');
```

Posts

SQL

```
INSERT INTO posts (title, content, author_id, published) VALUES
('Introducción a Node.js', 'Node.js es un runtime de JavaScript...', 1, true),
('PostgreSQL vs MySQL', 'Ambas bases de datos tienen ventajas...', 2, true),
('APIs RESTful', 'REST es un estilo arquitectónico...', 1, true),
('Manejo de errores en Express', 'El manejo apropiado de errores...', 3, false),
('Async/Await explicado', 'Las promesas simplifican el código asíncrono...', 1, false);
```

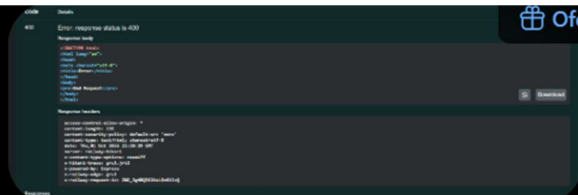
Y podemos agregar comentarios

Como ahora ya existen autores y posts:

SQL

```
INSERT INTO comments (content, author_id, post_id) VALUES
('Excelente explicación de Node.js.', 2, 1),
('Muy buen artículo.', 3, 1),
('Me resultó muy útil la comparación.' 1, 2);
```

+ Pregunta lo que quieras



Proyecto M2.txt
Documento

por que me sale este codigo http en el swagger cuando no ingreso un json valido por ejemplo:

```
{
  "author_id":yyy,
  "title": "Mi primer post",
  "content": "Este es el contenido de mi primer post",
  "published": true
}
```

autor_id es integer pero si ingreso yyy me sale el codigo http en el swagger

Sí, en tu caso es normal que aparezca ese `400 Bad Request`, y hay una diferencia importante entre JSON inválido y JSON válido pero con un dato incorrecto.

Agregá este middleware después de `express.json()` :

`</> JavaScript`

```
app.use(express.json());
```

```
app.use((err, req, res, next) => {
  if (err instanceof SyntaxError && err.status === 400 && err.type === "entity.parse.failed") {
    return res.status(400).json({
      msg: "El JSON enviado no es válido"
    });
  }

  next(err);
});
```

Categoría	Requisitos de la categoría	ENTRADA	ENTRADA/PROCESO	RESULTADO
1. Seguridad y privacidad de los datos	Implementar políticas de seguridad de datos (GDPR, etc.) y garantizar la privacidad de los datos de los usuarios.	Realizar una auditoría de seguridad de los datos de los usuarios y garantizar la privacidad de los datos de los usuarios.	CRUD de usuarios con políticas de seguridad de los datos de los usuarios.	Políticas de seguridad de los datos de los usuarios.
2. Autenticación y autorización de usuarios	Implementar un sistema de autenticación y autorización de usuarios.	Implementar un sistema de autenticación y autorización de usuarios.	Base de datos con datos de usuarios y contraseñas.	Usuarios autenticados y autorizados.
3. Integración de servicios externos	Integrar servicios externos (APIs, etc.) y garantizar la interoperabilidad de los datos.	Quitar los datos de los servicios externos y garantizar la interoperabilidad de los datos.	Integración de servicios externos (APIs, etc.) y garantizar la interoperabilidad de los datos.	Quitar los datos de los servicios externos y garantizar la interoperabilidad de los datos.
4. Validación y manejo de errores	Implementar un sistema de validación y manejo de errores.	Validación de los datos de los usuarios y manejo de los errores.	Validación de los datos de los usuarios y manejo de los errores.	Validación de los datos de los usuarios y manejo de los errores.
5. Testing automatizado	Implementar un sistema de testing automatizado.	Implementar un sistema de testing automatizado.	Implementar un sistema de testing automatizado.	Implementar un sistema de testing automatizado.
6. Documentación y mantenimiento	Implementar un sistema de documentación y mantenimiento.	Implementar un sistema de documentación y mantenimiento.	Implementar un sistema de documentación y mantenimiento.	Implementar un sistema de documentación y mantenimiento.



M2 MANZANO CESAR LUIS.txt

Documento

tengo que entregar este proyecto para que me corrija el evaluador con los criterios de la imagen te puedes pensar como un desarrollador señor así me decis si mi proyecto estaría aprobado o no, o que me faltaría ya anteriormente me corrigieron pero me faltó algunas cosas y desaprobe así que ahora quiero que me ayudes esto me contesto pero desde ahí lo mejoré y quise entregar un mejor proyecto: ¡Hola Cesar! Se puede observar una buena base técnica en la integración entre Express y PostgreSQL, especialmente por el uso de queries parametrizadas, las validaciones de tipos, el manejo diferenciado de errores de PostgreSQL y la incorporación de tests reales que fueron ejecutados contra una

Ver más ▾

tengo que entregar este proyecto para que me corrija el evaluador con los criterios de la imagen te puedes pensar como un desarrollador señor así me decís si mi proyecto estaría aprobado o no, o que me faltaría ya anteriormente me corrigieron pero me faltó algunas cosas y desaprobé así que ahora quiero que me ayudes esto me contestó pero desde ahí lo mejoré y quise entregar un mejor proyecto: ¡Hola César! Se puede observar una buena base técnica en la integración entre Express y PostgreSQL, especialmente por el uso de queries parametrizadas, las validaciones de tipos, el manejo diferenciado de errores de PostgreSQL y la incorporación de tests reales que fueron ejecutados contra una

[Ver más](#) ▾

Procesó durante 35s >

Sí, César. **Esta vez hice una revisión bastante más exigente**, tomando la rúbrica de la imagen como si yo fuera el evaluador y comparándola con el contenido real de tu entrega que me pasaste.

Mi conclusión corta es:

● Yo lo consideraría **APROBABLE** actualmente, pero **no lo entregaría todavía sin corregir 4 detalles concretos**.

Si corriges esos puntos, tu proyecto queda mucho más sólido y reduces bastante el riesgo de que el evaluador vuelva a encontrar algo para desaprobarte.